

Post-Mortem Analysis of the Automated Generation Pipeline

Comparison between `finApp` (manual implementation) and the project generated by GitHub Copilot (`AppFromPrompts`)

NECKER

July 12, 2026

Abstract

This document analyzes the outcome of a software development experiment entirely driven by AI agents: the reconstruction of a financial web application (a trading simulator with social groups) starting exclusively from the `specifich.md` document, using a sequential prompting pipeline fed into GitHub Copilot. The generated result (`Desktop/prompts` folder) is compared against the reference manual implementation (`finApp`). The analysis quantifies what worked (perfect type-safety, broad functional coverage, process discipline), what failed (81 out of 265 test suites failing, triple implementation of the same business logic, a queue-based architecture built but never connected, makeshift integration at the end of the project) and reconstructs the root causes, with particular focus on the mechanisms of *context rot* — the progressive degradation of coherence when multiple agents work in isolation on partial contexts. The document concludes with operational recommendations for the next iteration of the pipeline.

Contents

1	Introduction and Objective	3
2	The Pipeline Used	3
2.1	Phase 1 — Expansion of Specifications into Four Technical Documents	3
2.2	Phase 2 — Decomposition of Each Document into Atomic Features	3
2.3	Phase 3 — Consistency Check of Feature Documents	4
2.4	Phase 4 — Creation of the MASTER_SYSTEM_CONTEXT	4
2.5	Phase 5 — Implementation of Each Feature (One Agent per Feature)	4
2.6	Phase 6 — QA per Feature (Gap Analysis)	4
2.7	Phase 7 — Integration Testing per Document	4
3	Analysis Methodology	4
4	State of the Generated Project: The Metrics	5
4.1	Overview	5
4.2	Compilation: Green	5
4.3	Tests: Red, despite what the summaries state	5
4.4	Commit Timeline	5
5	What Worked	6
6	What Went Wrong	6
6.1	The Triple Implementation of Business Logic	6

6.2	The Queue Architecture: Built, Tested, Never Connected	7
6.3	The API Monolith and the App Running on Mocks	7
6.4	The Breakdown of Failed Tests	8
6.5	RBAC: The Middleware Exists, the Routes Do Not Use It	8
6.6	Document Expansion and Invented Features	8
6.7	Anomalous Project Structure	9
6.8	Unequal QA Coverage	9
7	Comparison with finApp	9
7.1	Opposing Philosophies	9
7.2	Key Observations	10
8	Root Cause Analysis: Where Context Rot Struck	10
8.1	Cause 1 — Isolation of Agents Without Shared Executable Contracts	10
8.2	Cause 2 — Integration as a Final Phase Rather Than a Continuous Constraint .	10
8.3	Cause 3 — Local "Green" Mistaken for Global "Green"	10
8.4	Cause 4 — Document Expansion as an Hallucination Multiplier	11
8.5	Cause 5 — Global Rules Losing Against Local Instructions	11
9	Recommendations for the Next Pipeline Iteration	11
10	Conclusions	12
A	Appendix A — Full Test Suite Outcome	13
B	Appendix B — Duplication Inventory	13
C	Appendix C — Essential Timeline of the Generated Project	14

1 Introduction and Objective

The experiment stems from a precise question: *is it possible to recreate a real, already hand-built web app using exclusively a pipeline of structured prompts and coding agents, without falling into the classic errors of context rot?*

The two compared artifacts are:

- **finApp** — the reference manual implementation: Express + Prisma + PostgreSQL backend, Vite + React frontend (SPA with React Router), images hosted on Cloudinary, trading engine driven by `setInterval`. Approximately 16,800 lines of TypeScript, 30 commits between March 19 and April 4, 2026.
- **Desktop/prompts** — the project generated by the pipeline: a monorepo with an Express + Prisma backend and a Next.js (App Router) frontend nested inside `src/frontend`, approximately 23,500 lines of source code plus 26,400 lines of tests (265 test files), 384 commits between February 22 and March 19, 2026.

The concise, anticipated verdict: **the pipeline produced a huge amount of code of good local quality, but not an end-to-end integrated and working application**. Each feature, taken on its own, is well-written, typed, and tested; however, the system as a whole contains three parallel implementations of the same business logic, an entirely dead queue infrastructure, and an improvised final integration phase that broke previously green tests without any agent noticing. Context rot did not manifest as "the agent forgets instructions", but as a **systemic divergence between parallel worlds built by agents that never spoke to each other**.

2 The Pipeline Used

The pipeline was composed of seven types of prompts, applied sequentially. A detailed description is provided here, as the very structure of the pipeline is part of the root causes analyzed later.

2.1 Phase 1 — Expansion of Specifications into Four Technical Documents

A prompt with the role of "Senior Software Architect and Tech Lead" receives `specifich.md` (327 lines) and generates four "progressive" technical documents: (1) architecture and database construction, (2) backend engine and core algorithms, (3) API infrastructure and security, (4) frontend and UI/UX. The fundamental rule imposed was: *"DO NOT SUMMARIZE. Each document must be long, extremely detailed, and include every single business rule"*.

Result: 4 documents for a total of **11,154 lines** (1,337, 3,585, 2,329, and 3,903 lines respectively) — an expansion factor of roughly 34× compared to the original specification.

2.2 Phase 2 — Decomposition of Each Document into Atomic Features

Each document is broken down into "independent atomic features", each with its own specification document designed as a system prompt for a TDD agent. Key requirements: strict Red-Green-Refactor cycle, Conventional Commits at every green cycle, and above all *"each document must allow an agent to work without knowing about the other features"*.

Result: **42 total features** — 12 for the database document, 12 for the backend, 10 for the APIs, 8 for the frontend.

2.3 Phase 3 — Consistency Check of Feature Documents

A verification prompt asked whether, by executing all features with one agent each, a complete application would be obtained in an orderly and coherent manner. For document 4, this phase successfully identified real issues (the `GroupCard` component was duplicated across three features, undeclared dependencies between features 3, 5, and 8) and produced two corrective artifacts: `FIX_REPORT.md` and `SHARED_CONTRACTS.md`, the latter containing shared TypeScript contracts. It is significant that these artifacts exist *only* for document 4: for documents 1–3, the verification left no equivalent traces.

2.4 Phase 4 — Creation of the `MASTER_SYSTEM_CONTEXT`

A synthesis prompt condensed specifications + 4 documents into a single context file for the agents (`MASTER_SYSTEM_CONTEXT.md`, 329 lines): non-negotiable business rules, roles/permissions matrix, essential DB schema, endpoint references, trading engine flow, and the technology stack (with an explicit ban on Next.js Pages Router), exact error codes and text, and DO/NEVER guidelines — including the capitalized rule on **mandatory mocking** of queues, Redis, and external APIs in tests.

2.5 Phase 5 — Implementation of Each Feature (One Agent per Feature)

Each agent received as their "entire universe of knowledge" exclusively their own feature file and the master context, with explicit instructions to ignore any other convention. They had to commit at the baseline (to isolate features), then at every green test, self-verify at the end of the work, and produce a summary file (`RESOCONTO_FEATURE_*.md`).

2.6 Phase 6 — QA per Feature (Gap Analysis)

A prompt with the role of "Senior QA Engineer" compared the implementation against the PRD document, producing a diagnostic report categorized by severity (BLOCKER / MAJOR / MINOR) regarding gaps, domain constraints, RBAC, and edge cases.

2.7 Phase 7 — Integration Testing per Document

Once all features of a document were completed, an "SDET" prompt generated an integration suite (Jest + Supertest + Prisma) focused on flow isolation, domain errors, RBAC, and mocking of external services. Each phase was followed by eventual correction cycles if the checks failed.

3 Analysis Methodology

The analysis was conducted directly on the two repositories, using reproducible metrics:

- **Compilation:** running `npm run tsc --noEmit` on the entire generated project.
- **Testing:** full execution of `npm run jest` (265 suites), categorizing failures by cause (environmental error vs real regression) through log inspection.
- **Git History:** temporal distribution of the 384 commits, adherence to Conventional Commits, and the "commit on every green" pattern.
- **Structural Analysis:** mapping services, repositories, workers, middlewares, and their relative import dependencies to detect dead code and duplications.
- **Document Comparison:** original specifications vs generated documents vs code, and process artifacts (`TODO.md`, `VERIFICA_SPECIFICHE.md`, diagnostic reports, feature summaries).

- **Comparison with finApp:** Prisma schema, runtime architecture, frontend structure, cron/image management.

4 State of the Generated Project: The Metrics

4.1 Overview

Metric	prompts (generated)	finApp (manual)
Source TypeScript lines	~23,556	~16,817
Test lines	~26,391 (265 files)	0
Commits	384 (Feb 22 – Mar 19)	30 (Mar 19 – Apr 4)
REST Endpoints	68 (in a single file)	~60 (by domain, 5 routers)
Prisma Migrations	12	13
Prisma Models	11 + 5 enums	12 + 5 enums
DB Triggers	subset (in 6 migrations)	dedicated migration
Frontend	Next.js App Router in <code>src/frontend</code>	Vite + React Router, separate repo
Order Queue	BullMQ/Redis (never connected)	<code>setInterval</code> 60s

Table 1: Quantitative comparison between the two projects.

4.2 Compilation: Green

`tsc -noEmit` terminates with **0 errors** out of approximately 50,000 total lines. This is a non-trivial result and must be acknowledged: the strict TypeScript constraint imposed by the master context held up across 42 different agents.

4.3 Tests: Red, despite what the summaries state

The full execution of the test suite outputs:

81 suites failed / 183 passed / 1 skipped — **370 tests failed** / 589 passed out of 963

Every single `RESOCONTO_FEATURE_*.md` file instead declares its own suite as green, and the diagnostic report of document 1 (dated March 17) certifies "75 passed, 1 skipped". This gap between the *declared state* and the *real state* is the single most important piece of data in the entire analysis, and it is broken down in Section 6.4.

4.4 Commit Timeline

The chronological distribution tells the real story of how the work progressed:

Day	Commits	Prevalent Activity
Feb 22–23, 2026	14	Frontend features (doc 4): profile, images, first Next.js skeleton
Mar 13, 2026	18	Resumption: friendship DB guards (doc 1)
Mar 16, 2026	23	Doc 1 features (stocks, portfolio, transactions)
Mar 17, 2026	200	Bulk of features for docs 1–3 + gap closure
Mar 18, 2026	128	Remaining features, wiring, TODO remediation
Mar 19, 2026	1	Final touch

Table 2: Commits per day in the generated repository.

Half of the project (328 commits out of 384) was generated in 48 hours. In those same 48 hours, all integration decisions were made — and that is exactly where the damage is concentrated.

5 What Worked

It is important to be fair about the successes, because they define what to *keep* in the pipeline.

1. **Total Type-Safety.** Zero compilation errors out of 50k lines written by 42 different agents. The "TypeScript + Prisma everywhere" constraint in the master context worked as an implicit contract more effective than any prose.
2. **Data Schema Fidelity.** The generated Prisma schema is substantially identical to that of `finApp`: same entities (User, Group, Portfolio, Transaction, Holding, Friendship, Membership/Group Invitation, Watchlist, Historical Data), same enums, and systematic, correct use of `Decimal` for monetary values (never `Float`). The rule "DECIMAL, never float" — repeated in specifications, the master context, and individual features — was respected everywhere. **Targeted redundancy on critical rules pays off.**
3. **Frontend Functional Coverage.** The Next.js frontend covers all pages mentioned in the specification: public homepage, login/registration, personal dashboard, stock page with trading panel, groups area with detail views, social interface, user overview, and account settings. Dynamic routes (`groups/[id]`, `stock/[id]`, `user/[id]`) are correct, the ban on Pages Router was respected, Zustand and React Query are used as specified, and smart polling for pending orders exists (`useSmartPolling.ts`).
4. **Process Discipline.** The 384 commits are legitimately Conventional Commits, truly granular (one green cycle $\approx$ one commit), with a baseline `chore`: at the start of each feature. The git history is legible and allows the reconstruction of the build order — it is precisely thanks to this discipline that this analysis was possible.
5. **Database-Level Triggers and Constraints (Partial).** The migrations contain real triggers: auto-sorting and uniqueness of friendships, anti self-friendship constraints, automatic deletion of invitations upon joining a group, single-owner constraints, and check constraints on Buy/Sell consistency. The QA cycle of document 1 added a trigger on trading permissions and anti-bypass checks using `IS NOT NULL` during the "gap closure" phase.
6. **The QA Cycle Found Real Problems.** The diagnostic report for doc 1 correctly identified: missing triggers, IDOR risks on order creation (no verification of portfolio ownership), Admin/Owner RBAC divergence on balance modifications, and edge cases on canceling sell orders. The initial fixes were applied and tested. Phase 3 (feature consistency for doc 4) also intercepted the triplication of `GroupCard` *before* implementation, producing `SHARED_CONTRACTS.md`.
7. **The Final TODO as a Radiography.** `TODO.md` — generated during the final alignment phase — is a high-quality document: it accurately lists mocks to be replaced, wrong endpoints, missing client functions, architectural violations (charts that should have been delegated to TradingView), and logic bugs (loss of average price when canceling a sale). Almost everything was subsequently fixed and checked off.

6 What Went Wrong

6.1 The Triple Implementation of Business Logic

This is the most severe and instructive failure. Within the project, **three parallel implementations of the same concepts** co-exist (Table 3):

The two "Portfolio" services share absolutely nothing: one uses `Prisma.Decimal` and the error codes from `types/errors.ts`, while the other uses `decimal.js` and its own error hierarchy (`PortfolioServiceErrors`), along with invented concepts such as `DEFAULT_GROUP_BUDGET_LIMIT`.

Origin	Convention	Artifacts
Doc 1 features (database)	Italian, <code>nome.service.ts</code>	<code>portfolio.service.ts</code> , <code>transazione.service.ts</code> , 11 repositories in <code>src/repositories/</code> (persona, portafoglio, ...)
Doc 2 features (backend)	English, <code>PascalCase.ts</code>	<code>PortfolioService.ts</code> , <code>TradingService.ts</code> , 4 repositories in <code>src/database/repositories/</code>
Final wiring (TODO phase)	Inline, direct prisma queries	<code>runtimeDependencies.ts</code> : 1,317 lines that reimplement orders, balances, and snapshots using direct Prisma queries

Table 3: The three co-existing implementations of the same logic.

Neither is wrong on its own — they are *coherent but incompatible worlds*, built by agents who were explicitly commanded to ignore everything outside their own feature file. The problem is also acknowledged internally: the last open item in `TODO.md` reads "*There is redundant or non-unified logic between `TransazioneService.executeOrder` and `TransactionWorker.ts`*".

6.2 The Queue Architecture: Built, Tested, Never Connected

Features 1–3 of document 2 built the entire BullMQ/Redis infrastructure: `src/queue/` (connection, queues, scheduler, types), `TransactionWorker.ts` (366 lines), `SnapshotWorker.ts`, complete with green unit tests and mocks compliant with the master context rules. Then, at the moment of the actual wiring, `runtimeDependencies.ts` implemented the trading engine using two simple `setInterval` loops (order sweeps and snapshots), directly calling `TransazioneService.executeOrder`. As a result, **the entire queue infrastructure is dead code** — no runtime file imports the workers or the queues.

The irony is that choosing `setInterval` was the *right* decision: it is exactly what `finApp` does (an internal 60-second cron in `tradingEngine.ts`). The problem is not the final solution, but that the pipeline had agents build and test a complex infrastructure for days, which the integration phase then entirely ignored, without anyone removing or connecting it.

6.3 The API Monolith and the App Running on Mocks

`createApiApp.ts` is a file of **2,553 lines** containing all 68 endpoints. It defines its own dependency contract (`ApiDependencies`: `authService`, `portfolioService`, `tradingService`, `marketService`, `adminService`) which **does not correspond to any of the services built by the features of documents 1 and 2**: it is a third taxonomy, invented by the "API routes" feature agent. To make it run, `src/index.ts` provided `createDevDependencies()`: hardcoded fake data (a portfolio with AAPL and MSFT, a dev-access-token, search routes returning empty arrays).

In practice, **for most of the project's lifespan, the app "worked" on fake data**, and all HTTP tests passed against mocks. Only in the final stage (`TODO.md`, item 1) were the mocks replaced by the real bootstrap with Prisma — and it is in this late stitching that the chaos concentrated: the wiring phase had to uncover and fix dozens of discrepancies all at once (wrong paths, missing client methods, disconnected crons, snapshots without frozen assets).

6.4 The Breakdown of Failed Tests

The 81 failed suites split into two very distinct categories:

Category	Suites	Cause
Doc 1 feature tests (friendship, watchlist, user, transaction, stock, portfolio, permission, membership, invitation, holdings, historical, group: 6–7 suites each)	~73	<code>PrismaClientInitializationError</code> : tests require a live local PostgreSQL instance with local credentials (868 occurrences of "Authentication failed against database server"). Not reproducible outside the specific machine/moment they were written.
Frontend and integration tests (homepage it. 1, personal-dashboard it. 1/8/11, social-interface it. 5, doc4-remediation)	~8	Real regressions : subsequent wiring and remediation commits (e.g., 6b83485 "close feature3 wiring gaps") modified components — removing <code>testid</code> strings like <code>portfolio-overview-skeleton</code> , changing compositions — without re-running the suites of previous iterations.

Table 4: Classification of the 81 failed suites.

Both families represent *pipeline* failures, not bad luck:

- The first is a **direct violation of the most emphasized rule in the master context** ("MOCKING MANDATORY [...] tests must NEVER attempt real network connections"): the agents for document 1 systematically wrote integration tests against a live Postgres database. The capitalized rule was not enough, because their feature files (their "universe of knowledge") described DB triggers and constraints that *only make sense* on a real database. When local instructions and global rules conflict, the agent follows the local instructions.
- The second highlights the absence of a **regression gate**: no agent in the subsequent phases had the obligation to re-run the entire test suite before committing. Everyone only checked their own tests ("green"), and other people's breakages remained invisible. This is the perfect formalization of context rot: *whatever is outside the agent's context does not exist, even when it is a red test*.

6.5 RBAC: The Middleware Exists, the Routes Do Not Use It

Feature 2 of document 3 built `rbacGroupMiddleware.ts` + `PermissionSystem` with green tests. However, the routes in `createApiApp.ts` execute manual role checks instead (`getEffectiveGroupRole`) — an item still marked as open in the TODO. Same pattern as the queue: **the correct component exists, but the integration ignores it**. The "Admin expels Admin" flaw was eventually closed, but inside the service layer, creating yet another source of truth for permissions (middleware, service, inline checks).

6.6 Document Expansion and Invented Features

The ban on summarization in Phase 1 turned 327 lines of specifications into 11,154 lines of documents. In that $34\times$ expansion, each document *invented* plausible but unrequested requirements, which subsequent phases treated as contractual obligations, producing entire features:

- doc 2, feature 11: *Performance Monitoring* (`PerformanceCollector`, `PerformanceAnalytics`, `AlertManager`, `HealthCheckService`);

- doc 3, features 9–10: *Error Handling & Resilience* (circuit breakers, retries with backoff) and *Rate Limiting & Security* (ddosProtector, securityAuditLogger, adaptive rate limiter);
- doc 4, feature 7: user achievements, privacy settings, performance charts with timeframes — all absent from the specs;
- the master context added on its own the dual-API strategy "Finnhub real-time + Polygon.io daily" (the specs cited Finnhub *or* Polygon as alternatives), and an `AnalyticsWebSocketService` appeared in the code despite the specification explicitly prescribing short polling.

A significant portion of the ~50,000 generated lines — plausibly a quarter — serves requirements that do not exist. Every invented line cost generation time, testing, QA, and broadened the surface area where context rot could strike.

6.7 Anomalous Project Structure

A single `package.json` mixes Express, Prisma, BullMQ, Next.js, React 19, and testing libraries; the frontend lives in `src/frontend` *inside* the backend source tree (with the `.next` folder committed alongside the source code); tests are scattered across three different locations (`src/tests/`, `*.test.ts` files next to source files, and `__tests__/` folders). This is the accumulation of 42 local decisions never reconciled — no agent had the authority (or context) to say "the frontend belongs in a separate project". `finApp`, by comparison, has two independent packages with clean boundaries.

6.8 Unequal QA Coverage

The diagnostic cycle planned for Phase 6 is documented only for document 1: inside `reports/`, there is uniquely `REPORT_DIAGNOSTICO_DOC1_GAP_ANALYSIS.md` (with two gap closure cycles). For documents 2, 3, and 4, no equivalent reports exist in the repository. Unsurprisingly, the areas left without formal QA (API wiring, frontend) are precisely those where real regressions are concentrated.

7 Comparison with finApp

7.1 Opposing Philosophies

	finApp (manual)	prompts (generated)
Philosophy	Minimum viable that works	Maximum theoretical described in docs
Trading Engine	<code>setInterval</code> 60s + endpoints secured by <code>CRON_SECRET</code>	BullMQ/Redis (dead) + <code>setInterval</code> (attivo)
Frontend	Vite SPA, React Router, separate repo	Next.js App Router inside the backend
Images	Cloudinary with client-side cropping	<code>OptimizedImage</code> + generative SVG placeholders (no real provider connected)
Testing	0 (manual verification + <code>API_ROUTES_CHECKLIST.md</code>)	963 (38% failing)
Security	JWT + auth/cron middleware, helmet, rate limit	JWT + RBAC + audit + DDoS protector (partial integration)
Source of Truth	The code itself + checklists	5 documents + 42 features + master context

Table 5: Architectural and methodological comparison.

7.2 Key Observations

1. The spec was more ambitious than the real app — and the pipeline could not know it. `specifich.md` describes Redis, BullMQ, multi-level caching, and three WebP image formats. `finApp` implements almost none of this: no Redis, no queues, simple crons. The person who wrote the spec knew which parts were aspirational; the pipeline, by design ("do not omit anything"), treated them all as binding requirements and further amplified them. **A specification document fed to an AI pipeline must distinguish "must-haves" from "nice-to-haves"**, because the AI lacks the context to do it on its own.

2. Where the spec was precise, the generated code aligned. Data schema, text error messages, pending/executed order flows, and private/group isolation: on everything the specs defined in detail, the two projects converge remarkably well. The quality of the output is directly proportional to the precision of the input.

3. The generated app is "more complete" on paper, less functional in practice. The generated project has more endpoints, more declared DB constraints, tests, monitoring, and resilience. But `finApp` is an application that runs coherently end-to-end, while the generated code is a collection of excellent modules with fragile welds. For a trading simulator, the end-to-end integrity of the order→execution→balance flow is worth more than any circuit breaker.

8 Root Cause Analysis: Where Context Rot Struck

The symptoms described above stem from five structural issues in the pipeline.

8.1 Cause 1 — Isolation of Agents Without Shared Executable Contracts

The rule "each agent works only with their feature file + master context" was designed precisely to fight context rot: small, fresh, focused contexts. However, it eliminated the only barrier against divergence: *mutual awareness*. The master context described business rules and stack, but not the **concrete contracts**: filenames, service signatures, naming conventions, and folder structures. Every agent therefore invented their own taxonomy, and 42 taxonomies do not compose a system. The proof of the opposite exists in the project itself: where a shared contract existed (`SHARED_CONTRACTS.md` for doc 4; the Prisma schema for everyone), convergence followed.

8.2 Cause 2 — Integration as a Final Phase Rather Than a Continuous Constraint

The pipeline built all the pieces first and then welded them together (mocks in `index.ts` → final wiring). Every interface error remained hidden until all pieces were stacked on top of each other, and at that point, corrections happened under pressure, in an by now massive context, by agents who did not know the original features: this is the exact moment the third inline implementation (6.1) and the frontend regressions were born. In terms of context rot: **the longer integration is postponed, the more context is needed to perform it, and the less context is available.**

8.3 Cause 3 — Local "Green" Mistaken for Global "Green"

The TDD protocol required committing at every green build *of one's own feature*. No phase required executing everything. The summaries are therefore all sincerely green, while the project is globally red. Without an automated gate external to the agents (CI), self-certification has no evidentiary value: the agent reports what it sees in its context, and in its context, everything was truly green.

8.4 Cause 4 — Document Expansion as an Hallucination Multiplier

Every generative step from document→document adds plausible details not present in the source; the subsequent step treats them as requirements. Over four hops (specifications → documents → features → master context → code), inventions accumulated and *legitimized each other* (the rate limiting invented by doc 3 fed into the master context, making it mandatory for everyone). This is context rot in document form: the source of truth drifts further away with every copy.

8.5 Cause 5 — Global Rules Losing Against Local Instructions

The mandatory mocking rule, despite being written in block capitals in the master context, was ignored by 12 out of 12 features in document 1 — because their feature files required verifying triggers and constraints that only exist in a live database. When global rules contradict local tasks, the agent resolves the conflict in favor of the task. Cross-cutting rules must therefore be made *impossible to violate* (provided environments, fixtures, test scripts) rather than just *forbidden in prose*.

9 Recommendations for the Next Pipeline Iteration

1. **Replace Expansion with Structuring.** Do not ask for four "long and exhaustive documents that omit nothing": ask for documents that *reorganize* specifications without adding requirements, enforcing an inverse rule ("if a requirement is not in the specs, it cannot appear; if you feel something is missing, list it under an OPEN QUESTIONS section"). Explicitly mark in the specs what is binding and what is aspirational.
2. **Contracts Before Features (Contract-First).** Before launching any implementation agent, generate and freeze: the Prisma schema, an endpoint list with request/response DTOs (e.g., OpenAPI), **TypeScript interfaces of services with filenames**, folder structure, and a single naming convention (one language only). The `SHARED_CONTRACTS.md` from doc 4 must be promoted from a patch to a mandatory phase for the entire system. Every feature implements against these contracts, making compilation a free integration test.
3. **Walking Skeleton Before Features.** First agent task: build a minimal, real end-to-end working application (a user registers, buys a stock, the cron executes it, the balance updates — real DB, zero mocks in the runtime). All subsequent features mount onto a system that already works, making any integration breakage visible immediately. Explicitly ban "convenience" dependency injection with hardcoded mocks in the main entry point.
4. **Mechanical Regression Gate.** Every agent, before every commit, must execute *the entire* suite (or at least a centrally defined subset of end-to-end smoke tests), not just their own tests. Ideally: an `npm run verify` script provided by the skeleton, whose green status is a condition for the commit. Self-certification in summaries must be replaced by the actual output of the command.
5. **Two Test Categories with Provided Infrastructure.** Separate `test:unit` (mocked, always runnable) from `test:integration` (ephemeral PostgreSQL via Docker/testcontainers, launched by the script itself). The rule "never real connections in unit tests" is enforced by providing the setup template, not by writing it in capitals.
6. **A Recurring Integration Agent.** Every *N* features (not at the end), deploy an agent with an exclusive task: run the full suite, look for symbol/service duplications, verify that every new module is imported by the runtime (code unreachable from `index.ts` is an error), and update contracts if necessary. This would have caught both the double service layer and the dead queue weeks in advance.

7. **Uniform and Tracked QA.** The gap-analysis cycle must be executed (and archived in reports/) for every document, not just the first; and every fix post-QA must pass through the regression gate, because it is statistically the phase that introduces the most regressions.
8. **Size Features on Value, Not Decomposability.** 42 features for this app mean 41 stitching points. Decomposing into 10–15 *vertical* slices (e.g., "complete order lifecycle, DB→API→UI") reduces internal interfaces, which are exactly where context rot strikes.
9. **The Master Context Must Contain Normative Examples, Not Just Rules.** A reference file for a service, a test, and a route (with naming, imports, structure) constrains agents more than ten rules in prose: agents imitate much better than they obey.

10 Conclusions

The pipeline proved that a team of agents can produce, in a few days, the complete surface area of a non-trivial application with high local quality: correct types, faithful data schema, complete UI, exemplary git history. However, it also demonstrated, with quantitative evidence, that **local quality does not automatically assemble into global quality**: 38% of tests failed, three implementations of the same logic co-exist, a queue infrastructure is entirely dead, and a final integration phase broke what was previously green.

Context rot did not strike where it was expected — inside the single agent’s context, which the small-and-fresh context strategy actually protected well — but **in the spaces between agents**: in interfaces never agreed upon, in requirements multiplied at every document transition, in local "green" states never verified globally, and in integration postponed until the last minute. Fixing the pipeline therefore does not require better prompts, but **shared mechanical constraints**: executable contracts defined beforehand, an immediately working skeleton, and a regression gate that no agent can self-certify. Words in context degrade; compilable contracts and executed tests do not.

A Appendix A — Full Test Suite Outcome

Execution of `npx jest` on the Desktop/prompts repository (July 12, 2026):

Test Suites: 81 failed, 1 skipped, 183 passed, 264 of 265 total
Tests: 370 failed, 4 skipped, 589 passed, 963 total

Suites failed due to real regression (not environmental):

- `src/tests/homepage/iteration1.homepage-layout.test.tsx`
- `src/tests/personal-dashboard/iteration1.dashboard-layout.test.tsx`
- `src/tests/personal-dashboard/iteration8.dashboard-composition.test.tsx`
- `src/tests/personal-dashboard/iteration11.page-wiring-actions-and-polling.test.tsx`
- `src/tests/social-interface/iteration5.social-dashboard-composition.test.tsx`
- `src/tests/integration/doc4-remediation.integration.test.ts`

Suites failed due to a dependency on a live PostgreSQL database (868 occurrences of *Authentication failed against database server*): all suites in folders under `src/tests/` related to document 1 features — `friendship-system`, `watchlist-management`, `user-management`, `transaction-processing`, `stock-management`, `portfolio-management`, `permission-management`, `membership-management`, `invitation-system`, `holdings-management`, `historical-data-management`, `group-management` — plus `integration/doc1-full-system` and `integration/gap-closure`.

B Appendix B — Duplication Inventory

- **Portfolio Services:** `src/services/PortfolioService.ts` (494 lines, `decimal.js`, own errors) vs `src/services/portfolio.service.ts` (118 lines, `Prisma.Decimal`) vs inline logic in `src/runtime/runtimeDependencies.ts`.
- **Trading Services:** `src/services/TradingService.ts` (310 lines) vs `src/services/transazione.service.ts` (183 lines) vs `src/workers/TransactionWorker.ts` (366 lines, never imported by runtime).
- **Repositories:** `src/repositories/` (11 files, Italian naming: `persona`, `portafoglio`, `gruppo`, ...) vs `src/database/repositories/` (4 files, English naming: `UserRepository`, `PortfolioRepository`, ...).
- **Order Execution:** `TransazioneService.executeOrder` (used by runtime) vs `TransactionWorker` (BullMQ queue, dead) — duplication acknowledged as an open item in `TODO.md`.
- **Permission Checks:** `rbacGroupMiddleware` (built, not used by routes) vs manual checks `getEffectiveGroupRole` in `createApiApp.ts` vs checks in `PermissionService` — three sources of truth.

C Appendix C — Essential Timeline of the Generated Project

Reconstruction from the git history (384 total commits):

Date	Commits	Event
Feb 22–23	14	Doc 4 features (frontend): user profile, image system; initial Next.js skeleton.
Mar 13	18	Resumption after three weeks: friendship system with DB guards (doc 1).
Mar 16	23	Stock, portfolio, and transaction management (doc 1).
Mar 17	200	Peak: completion of doc 1, gap analysis, and two gap closure cycles; doc 2 features (queues, workers, services); start of doc 3.
Mar 18	128	Doc 3 features (JWT, RBAC, endpoints, resilience, rate limiting), API integration, <code>TODO.md</code> remediation, real runtime wiring.
Mar 19	1	Final commit (homepage client). On the same day, manual development of <code>finApp</code> begins.

Table 6: Timeline of the generated repository.